

FDU 数据库 4. 存储与索引

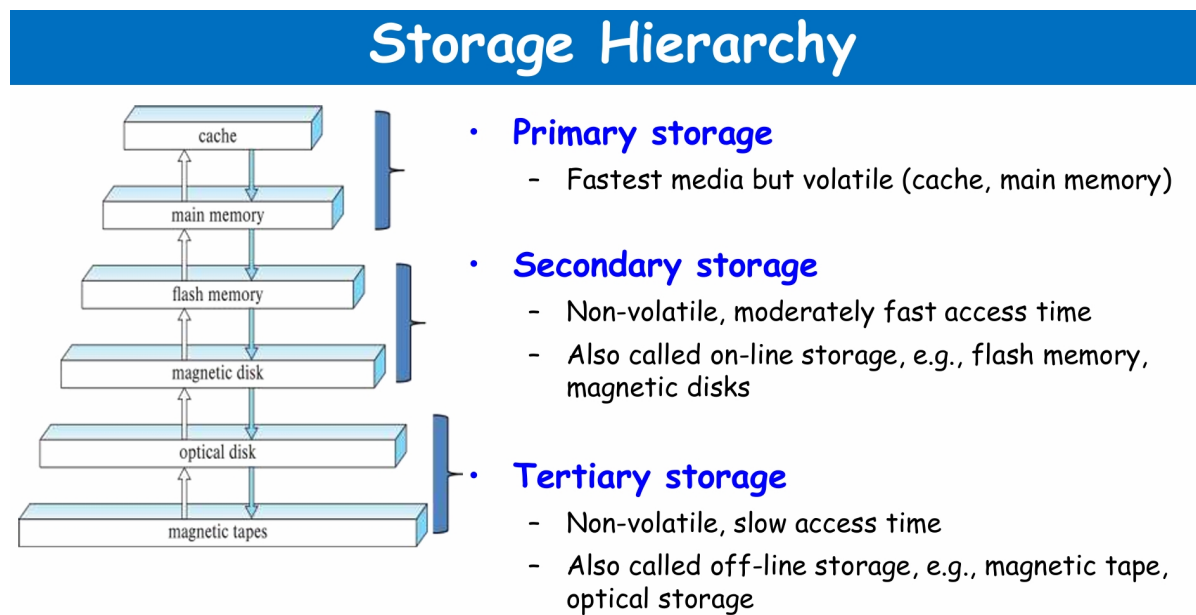
本文参考以下教材:

- Database System Concepts (A. Silberschatz, H. Korth, S. Sudarshan) Chapter 12, 13, 14
- 数据库系统概念 (A. Silberschatz, H. Korth, S. Sudarshan, 杨冬青等译) 第 12, 13, 14 章

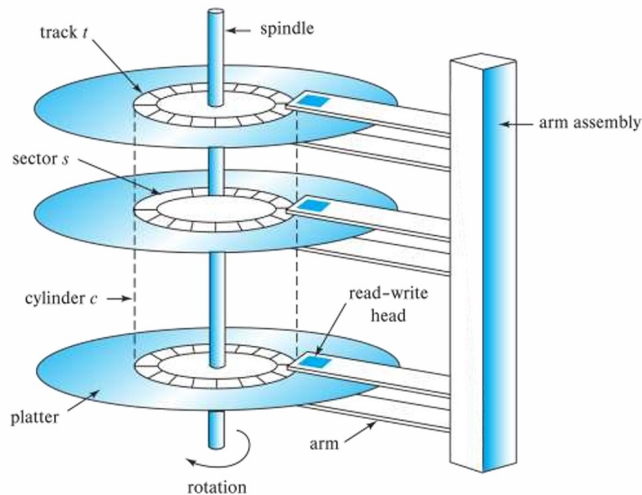
欢迎批评指正!

4.1 物理存储系统

具体内容参考 FDU 操作系统 4. 存储.



Magnetic Hard Disk Mechanism



Schematic diagram of magnetic disk drive

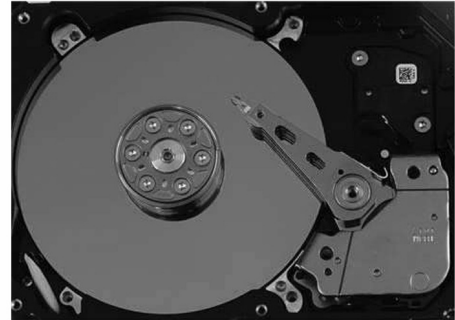


Photo of magnetic disk drive

- **Read-write head (读写头)**
 - positioned very close to the platter surface (almost touching it)
 - reads or writes magnetically encoded information
- **Surface of platter is divided into circular tracks (磁道)**
 - over 50K-100K tracks per platter on typical hard disks
- **Each track is divided into sectors (扇区)**
 - a sector is the smallest unit of data that can be read or written
 - sector size is typically **512 bytes**
 - typical sectors per track: 500-1000 (on inner tracks), 1000-2000 (on outer tracks)
- **To read/write a sector**
 - disk arm swings to position head on right track
 - platter spins continually; data is read/written as sector passes under head
- **Head-disk assemblies (磁头-磁盘装置)**
 - multiple disk platters on a single spindle (1 to 5 usually)
 - one head per platter, mounted on a common arm.
- **Cylinder(柱面) i consists of the i -th tracks of all the platters(盘片)**
- **Earlier generation disks were susceptible to head-crashes**
 - Surface of earlier generation disks had metal-oxide coatings (金属氧化物涂层) which would disintegrate on head crash and damage all data on disk
 - Current disks are less susceptible to such disastrous failures
- **Disk controller(磁盘控制器)** - interfaces between the computer system and the disk drive hardware
 - Accept high-level commands to read or write a sector
 - **Initiate actions** such as moving the disk arm to the right track and actually reading or writing the data
 - Compute and attach **checksums(校验和)** to each sector to verify that data is read back correctly.
 - Performs **remapping of bad sectors(坏扇区重映射)**

Performance Measures of Disks

- **Access time (访问时间)**

- the time it takes from when a read or write request is issued to when data transfer begins, including
 - **Seek time(寻道时间)** - time it takes to reposition the arm over the correct track.
 - Average seek time is about 1/2 the worst case seek time
 - 4 to 10 milliseconds 毫秒 on typical disks
 - **Rotational latency(旋转等待)** - time it takes for the sector to be accessed to appear under the head
 - Average latency is 1/2 of the worst case latency.
 - 4 to 11 milliseconds on typical disks (5400 to 15000 rpm)

- **Data-transfer rate (数据传输率)**

- the rate at which data can be **retrieved from or stored to the disk**
- Max rate: 25 to 100 MB per second, lower for inner tracks
- Multiple disks may share a controller, so rate that controller can handle is also important
 - E.g., ATA-5: 66 MB/sec, SATA: 150 MB/sec, Ultra 320 SCSI: 320 MB/s
 - Fiber Channel (FC2Gb): 256 MB/s

- **Mean time to failure (平均故障时间, MTTF)**

- the average time the disk is expected to run continuously without any failure, typically **3 to 5 years**
- Probability of failure of new disks is quite low, corresponding to a "theoretical MTTF" of **500,000 (~57 years) to 1,200,000 (~137 years)** hours for a new disk
 - E.g., an MTTF of 1,200,000 hours for a new disk means that given 1000 new disks, one will fail every 1200 hours on the average
- MTTF decreases as disk ages

Optimization of Disk-block Access

- **Block** - a contiguous sequence of sectors from a single track
 - data is transferred between disk and main memory in blocks
 - sizes range from 512 bytes to several kilobytes
 - small blocks: more transfers from disk
 - large blocks: more space wasted due to partially filled blocks
 - typical block sizes range from 4 to 16 kilobytes
- **Disk-arm-scheduling (磁盘臂调度) algorithms** order pending accesses to tracks so that disk arm movement is minimized
 - **elevator algorithm (电梯算法)** : move disk arm in one direction (from outer to inner tracks or vice versa), processing next request in that direction, till no more requests in that direction, then reverse direction and repeat
- **File organization** - optimize block access time by organizing the blocks according to the way of data access
 - E.g., store related information on the same or nearby cylinders
 - Files may get fragmented(碎片化) over time
 - data is inserted to/deleted from the file
 - free blocks on disk are scattered, and newly created file has its blocks scattered over the disk
 - sequential access to a fragmented file results in increased disk arm movement
 - Some systems have utilities to **defragment the file system** (去除文件碎片), in order to speed up file access
- **Non-volatile write buffers** speed up disk writes by writing blocks to a non-volatile **RAM buffer** immediately
 - Non-volatile RAM: battery backed up RAM or flash memory
 - Even if power fails, the data is safe and will be written to disk when power returns
 - **Controller writes to disk when the disk has no other requests or the non-volatile RAM is full**
 - Database operations can continue without waiting for data to be written to disk
 - Writes can be reordered to **minimize disk arm movement**
- **Log disk (日志磁盘)**
 - A disk devoted to **writing a sequential log of block updates**
 - Used exactly like nonvolatile RAM
 - Write to log disk is very fast since seeking is not required
 - No need for special hardware (NV-RAM)
- **File systems typically reorder writes to disk to improve performance**
 - **Journaling file systems (日志文件系统)** write data in safe order to NV-RAM or log disk
 - Reordering without journaling: risk of corruption of file system data

RAID (独立冗余磁盘阵列)

- **RAID: Redundant Arrays of Independent Disks**
 - Disk organization techniques that manage a large numbers of disks, providing a view of a single disk of
 - **High capacity** and **high speed** by using multiple disks in parallel
 - **High reliability** by storing data **redundantly**. Data can be recovered even if a disk fails
 - The chance that some disk out of a set of N disks will fail is much higher than the chance that a specific single disk will fail
 - E.g., a system with 100 disks, each with MTTF of 100,000 hours (approx. 11 years), will have a system MTTF of 1000 hours (around 41 days)
 - Techniques for using redundancy to avoid data loss are critical with large numbers of disks
 - **Originally a cost-effective alternative to large, expensive disks**
 - I in RAID originally stood for "**inexpensive**"
 - Now the "I" is interpreted as **independent**
 - **RAID(独立冗余磁盘阵列)**

Improvement of Reliability via Redundancy

- **Redundancy** - store extra information that can be used to rebuild information lost in a disk failure
 - **Mirroring (镜像)**: Duplicate every disk, and each logical disk consists of two physical disks
 - **Every write is carried out on both disks. Reads can take place from either disk**
 - If one disk in a pair fails, data still available in the other
 - Data loss would occur only if a disk fails, and its mirror disk also fails before the system is repaired
 - Probability of combined event is very small except for dependent failure modes such as fire or building collapse or electrical power surges
- **Mean time to data loss (平均数据丢失时间)** depends on **mean time to failure (平均故障时间)** and **mean time to repair (平均修复时间)**
 - $MTDL = MTF^2 / (2 * MTR)$ (why?)
- Two main goals of **parallelism** in a disk system
 - Load balance multiple small accesses to increase throughput
 - Parallelize large accesses to reduce response time
- **Bit-level striping (位级拆分)**: split the bits of each byte across multiple disks
 - In an array of eight disks, write bit i of each byte to disk i
 - Each access can read data at eight times the rate of a single disk
- **Block-level striping (块级拆分)**: with n disks, block i of a file goes to disk $(i \bmod n) + 1$
 - Requests for different blocks can run in parallel if the blocks reside on different disks
 - A request for a long sequence of blocks can utilize all disks in parallel

RAID Levels

- Schemes to provide **redundancy** at lower cost by using **disk striping (磁盘拆分)** combined with **parity bits(奇偶校验位)**

- Different RAID organizations, or RAID levels, have differing cost, performance and reliability characteristics

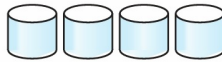
RAID Level 0: Block striping, non-redundant. (无冗余拆分)

Used in high-performance applications where data loss is not critical

RAID Level 1: Mirrored disks with block striping (镜像磁盘)

Offers best write performance.

Popular for applications such as storing log files in a database system

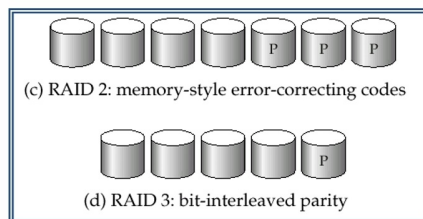


(a) RAID 0: nonredundant striping



(b) RAID 1: mirrored disks

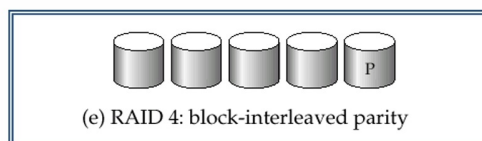
- RAID Level 2: Memory-Style Error-Correcting-Codes (ECC) with bit striping.** (内存风格的纠错码, 奇偶校验位)
- RAID Level 3: Bit-Interleaved Parity** (位交叉的奇偶校验)
 - a single parity bit is enough for **error correction**, not just detection, since we know which disk has failed
 - When writing data, corresponding parity bits must also be computed and written to a parity bit disk
 - To **recover** data in a damaged disk, compute XOR of bits from other disks (including parity bit disk)



(c) RAID 2: memory-style error-correcting codes

(d) RAID 3: bit-interleaved parity

- RAID Level 3 (Cont.): Bit-Interleaved Parity** (位交叉的奇偶校验)
 - Faster data transfer than with a single disk, but fewer I/Os per second since every disk has to participate in every I/O.
 - **Subsumes Level 2** (provides all its benefits, **at lower cost**).
- RAID Level 4: Block-Interleaved Parity** (块交叉的奇偶校验)
 - uses **block-level striping**, and **keeps a parity block on a separate disk** for corresponding blocks from N other disks.
 - When **writing data block**, corresponding block of parity bits must also be computed and written to parity disk
 - To **find value of a damaged block**, compute XOR of bits from corresponding blocks (including parity block) from other disks.

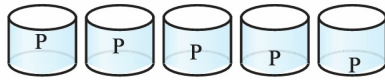


(e) RAID 4: block-interleaved parity

- RAID Level 4 (Cont.): Block-Interleaved Parity** (块交叉的奇偶校验)
 - Provides **higher I/O rates** for independent block reads than Level 3
 - block read goes to a single disk, so blocks stored on different disks can be read **in parallel**
 - Provides **high transfer rates** for reads of multiple blocks than no-striping
 - **Before writing a block**, parity data must be computed

Before writing a block, parity data must be computed

- Can be done by using old parity block, old value of current block and new value of current block (2 block reads + 2 block writes)
 - More efficient for writing large amounts of data sequentially
- Parity block becomes a bottleneck for independent block writes since every block write also writes to parity disk
- **RAID Level 5: Block-Interleaved Distributed Parity**(块交叉的分布奇偶校验)
 - partitions data and parity among all $N + 1$ disks, rather than storing data in N disks and parity in 1 disk.
 - E.g., with 5 disks, parity block for n th set of blocks is stored on disk $(n \bmod 5) + 1$, with the data blocks stored on the other 4 disks.



RAID 5: block-interleaved distributed parity

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

- **RAID Level 5 (Cont.) Block-Interleaved Distributed Parity**(块交叉的分布奇偶校验)
 - Higher I/O rates than Level 4.
 - Block writes occur in parallel if the blocks and their parity blocks are on different disks.
 - Subsumes Level 4: provides same benefits, but avoids bottleneck of parity disk.
- **RAID Level 6: P+Q Redundancy scheme**
 - similar to Level 5, but stores extra redundant information to guard against multiple disk failures.
 - Better reliability than Level 5 at a higher cost; not used as widely.



(d) RAID 6: P + Q redundancy

Choice of RAID Level

- **Factors in choosing RAID level**
 - Monetary **cost**
 - **Performance**: Number of **I/O** operations per second, and bandwidth during normal operation
 - Performance during **failure**
 - Performance during **rebuild** of failed disk
 - Including time taken to rebuild failed disk
- **RAID 0** is used only when data safety is not important
 - E.g. data can be recovered quickly from other sources
- **Level 2 and 4** never used since they are subsumed by **3 and 5**
- **Level 3** is not used anymore since bit-striping forces single block reads to access all disks, wasting disk arm movement, which **block striping (level 5)** avoids
- **Level 6** is rarely used since **levels 1 and 5** offer adequate safety for almost all applications
- So competition is **between 1 and 5** only
- **Level 1** provides much **better write performance** than **level 5**
 - **Level 5** requires at least 2 block reads and 2 block writes to write a single block, whereas **Level 1** only requires 2 block writes
 - **Level 1** preferred for high update environments such as log disks
- **Level 1** had **higher storage cost** than **level 5**
 - Disk drive capacities increasing rapidly (50%/year) whereas disk access times have decreased much less (x 3 in 10 years)
 - I/O requirements have increased greatly, e.g. for Web servers
 - When enough disks have been bought to satisfy required rate of I/O, they often have spare storage capacity, so there is often no extra monetary cost for Level 1!
- **Level 5** is preferred for applications with **low update rate**, and **large amounts of data**
- **Level 1** is preferred for all other applications

Storage Access

- **Storage of database file**

- partitioned into **fixed-length storage units** called **blocks**. Blocks are units of both storage allocation and data transfer
- Database system seeks to **minimize the number of block transfers** between the disk and memory. We can reduce the number of disk accesses by keeping as many blocks as possible in main memory

- **Buffer (缓存)**

- The portion of main memory available to store copies of disk blocks
- **Buffer manager**: subsystem responsible for allocating buffer space in main memory

- Programs call the **buffer manager** when they need a block from disk

- If the block is already in the buffer, buffer manager returns the address of the block in main memory
- If the block is not in the buffer, **the buffer manager**
 - **Allocates space** in the buffer for the block
 - **Replacing** (throwing out) some other block, if required, to make space for the new block.
 - Replaced block is **written back** to disk only if it was modified since the most recent time that it was written to/fetched from the disk
 - **Reads the block** from the disk to the buffer, and **returns the address** of the block in main memory to requester

Buffer-Replacement Policies

- **LRU (Least Recently Used) 最近最少使用**

- Most operating systems **replace the block least recently used**
- Idea behind LRU - use past patterns of block references as a predictor of future references
- LRU can be a bad strategy for certain access patterns involving repeated scans of data
 - For example: when computing the join of 2 relations **r** and **s** by a nested loops

```
for each tuple tr of r do
  for each tuple ts of s do
    if the tuples tr and ts match ...
```

- **Pinned block(被钉住的块)**

- memory block that is **not allowed to be written back** to disk

- **Toss-immediate (立即丢弃) strategy**

- free the space occupied by a block **as soon as** the final tuple of that block has been processed

- **Most recently used (MRU) (最近最常使用) strategy**

- system must **pin the block** currently being processed. After the final tuple of that block has been processed, the block is **unpinned**, and it becomes the most recently used block

4.2 文件组织

文件是操作系统为应用程序提供了一种抽象。

一个数据库被映射为多个不同的文件，这些文件由操作系统来维护。

一个文件在逻辑上被组织成一个序列，映射到若干个磁盘块上。

块是存储分配和数据传输的单位，块大小通常为 4KB 或 8KB。

一般来说，我们要求每条记录都被完全包含着单个块中，这样可以加快对数据项的访问。

但对于大数据项 (例如图像)，它们需要多个块存储，我们通过指针来记录使用了哪些块。

在关系数据库中，不同关系的元组通常具有不同的规模。

把数据库映射到文件的一种方法是使用多个文件，在任意给定的文件中只存储一种固定长度的记录。

另一种选择是自定义变长记录文件，使之能够容纳不同长度的记录。

前者的实现显然更简单，后者则更节省空间。

4.2.1 定长记录

考虑大学数据库中 instructor 关系的记录：

```
TYPE instructor = RECORD
    ID varchar (5);
    name varchar(20);
    dept name varchar (20);
    salary numeric (8,2);
END
```

每个元组需要 $5 + 20 + 20 + 8 = 53$ 字节。

我们在每个块中只分配它能完整容纳的记录的最多个数，余下的字节就不再使用了：

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.1 File containing *instructor* records.

当一条记录被删除时，我们可以把紧跟其后的记录移动到被删记录先前占据的空间中。

依此类推，直到被删记录后面的每条记录都向前做了移动 (见左图)。

然而这种方式需要移动大量的记录。

一种更简单的做法是将文件的最后一条记录移到被删记录所占据的空间中 (见右图)。

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Figure 13.2 File of Figure 13.1, with record 3 deleted and all records moved. **Figure 13.3** File of Figure 13.1, with record 3 deleted and final record moved.

移动记录以占据被删记录所释放的空间的做法并不理想，因为这样做需要额外的块访问。由于插入通常比删除更频繁，因此让被删除记录所占的空间空着，一直等到后面的插入操作重新使用这个空间，这样做是可以接受的。我们可在**文件头** (file header) 中使用链表来记录空缺的位置。

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

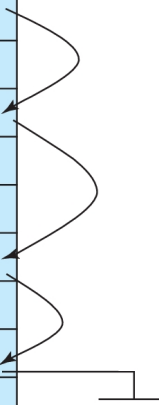


Figure 13.4 File of Figure 13.1, with free list after deletion of records 1, 4, and 6.

对定长记录文件的插入和删除很容易实现，因为被删除记录留出的空间恰好是插入记录所需要的空间。如果我们允许文件中的记录是变长的，这样的匹配将不再成立。

4.2.2 变长记录

(待补充)

4.3 顺序索引

4.3.1 基本概念

索引对于数据库中查询的高效处理至关重要。如果没有索引，那么每个查询都将读取它所使用的每个关系的全部内容。对于仅获取几条记录的查询来说，这样做的代价是不能接受的。

索引的基本类型:

- **顺序索引** (ordered index): 基于值的顺序排序

- **B⁺ 树索引** (B⁺-tree index): 基于 B⁺ 树 (一种平衡多路查找树), 特别适用于范围查询和动态更新场景.
- **散列索引** (hash index): 基于值将记录平均分布到若干桶中.
一个值所属的桶是由**散列函数** (hash function) 决定的.

索引的评价指标:

- **访问类型** (access type):
能有效支持的访问类型.
例如找到具有特定属性值的记录 (等值查询),
或是找到属性值落在某个特定范围内的记录 (范围查询),
或是找到具有特定属性值的记录以及最近的若干条查询 (*k*-邻近查询)
- **访问时间** (access time):
在查询中找到一个特定数据项或数据项集所花费的时间.
- **插入时间** (insertion time):
插入一个新数据项所花费的时间.
包括找到插入这个新数据项的正确位置所花费的时间, 以及更新索引结构所花费的时间
- **删除时间** (deletion time):
删除一个数据项所花费的时间.
包括找到待删除项所花费的时间, 以及更新索引结构所花费的时间.
- **空间开销** (space overhead):
索引结构所占用的额外空间.
倘若这种额外空间的规模适度, 通常值得牺牲一定的空间来换取性能的提升.

我们通常希望在一个文件上建立多个索引, 例如我们可能需要按作者、主题或书名来查找一本书.
用于在文件中查找记录的属性或属性集被称为**搜索键** (search key).
如果一个文件上有多个索引, 那么它就有多个搜索键.

主索引 (primary index) 又称**聚集索引** (clustering index),
其搜索键 (常常但并不总是主键) 指定的顺序与文件的排列顺序相同.

辅助索引 (secondary index) 或**非聚集索引** (nonclustering index),
其搜索键指定的顺序与文件的排列顺序不同.

本节中我们默认所有文件都按照某种搜索键顺序存储 (如下图所示).
这种在搜索键上有聚集索引的文件称作**索引顺序文件** (index-sequential file)
它们代表了在数据库系统中最早采用的索引模式之一,
是针对既需要顺序处理整个文件又需要随机访问单条记录的应用而设计的.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Figure 14.1 Sequential file for *instructor* records.

4.3.2 稠密索引 & 稀疏索引

索引项 (index entry) 由一个搜索键值和指针构成，这些指针指向具有该搜索键值的某些记录。
我们可以使用的顺序索引有两类：

- **稠密索引** (dense index):

在稠密索引中，文件中的每个搜索键值都有一个索引项。

在稠密聚集索引中，索引记录包括搜索键值以及指向具有该搜索键值的第一条数据记录的指针。

具有相同搜索键值的其余记录会顺序地存储在第一条记录之后。

在稠密非聚集索引中，索引必须存储指向具有相同搜索键值的所有记录的指针列表。

10101		→	10101	Srinivasan	Comp. Sci.	65000	
12121		→	12121	Wu	Finance	90000	
15151		→	15151	Mozart	Music	40000	
22222		→	22222	Einstein	Physics	95000	
32343		→	32343	El Said	History	60000	
33456		→	33456	Gold	Physics	87000	
45565		→	45565	Katz	Comp. Sci.	75000	
58583		→	58583	Califieri	History	62000	
76543		→	76543	Singh	Finance	80000	
76766		→	76766	Crick	Biology	72000	
83821		→	83821	Brandt	Comp. Sci.	92000	
98345		→	98345	Kim	Elec. Eng.	80000	

Figure 14.2 Dense index.

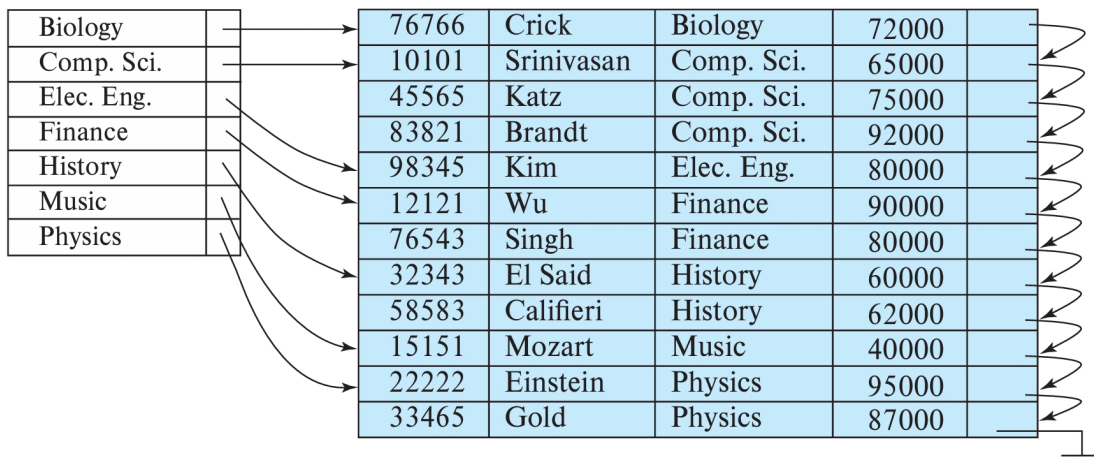


Figure 14.4 Dense index with search key *dept_name*.

- **稀疏索引 (sparse index):**

只有聚集索引才能使用稀疏索引.

在稀疏索引中, 只为某些搜索键值建立索引项.

每个索引项包括一个搜索键值和指向具有该搜索键值的第一条数据记录的指针.

为了定位一条记录, 我们找到搜索键值不超过我们所找记录的搜索键值的最大的索引项.

我们从该索引项指向的记录开始, 然后沿着文件中的指针查找, 直到找到所需记录为止.

例如字典可以在每页页眉列出该页中按字母顺序出现的第一个单词,

这样字典的每页顶部的单词共同构成了字典页面内容的一个稀疏索引.

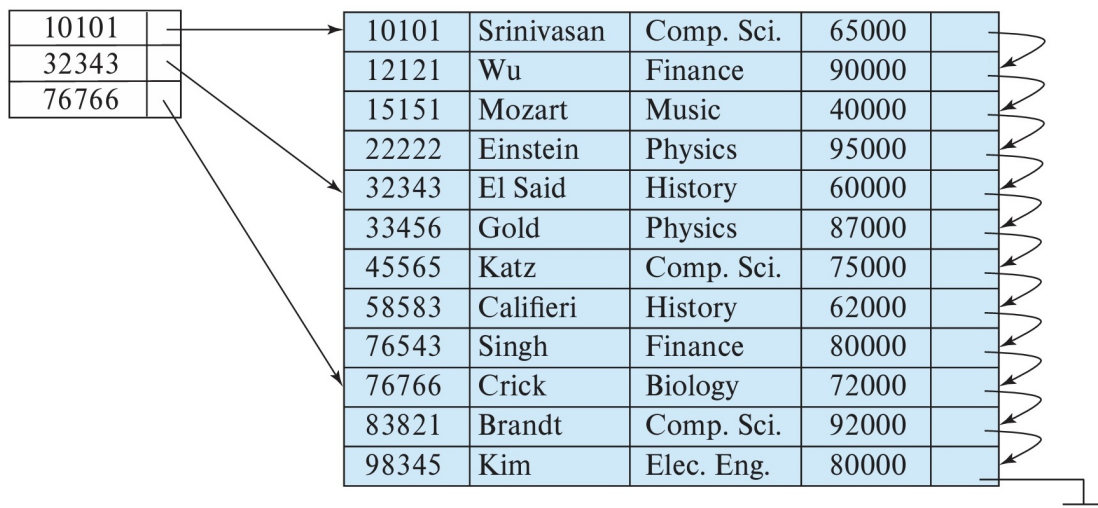


Figure 14.3 Sparse index.

利用稠密索引通常可以比稀疏索引更快地定位一条记录.

但稀疏索引占用的空间较小, 并且在插入和删除记录时的维护开销也较小.

我们必须在查询速度和空间开销 (以及维护开销) 之间做权衡.

为每个块建一个索引项的稀疏索引是一种好的折中方案.

原因在于, 处理一个数据库查询的开销主要由把块从磁盘读到主存中所花费的时间来决定.

一旦我们将块放入主存, 扫描整个块的时间就是可以忽略的.

使用这样的稀疏索引, 可以定位包含所要查找的记录的块.

只要记录不横跨多个块, 就能使块访问次数最小, 同时能保持索引的空间开销和维护开销尽可能小.

4.3.3 多级索引

如果索引足够小且可以完全放在主存中，那么查找一个索引项的搜索时间就会很短。但如果索引过大而不能完全放在主存中，那么查找一个索引项就需要多次读取磁盘块。为了处理这个问题，我们像对待其他顺序文件那样对待索引，在原始的索引（现在称为内层索引）上构造一个稀疏的外层索引（如图所示）。

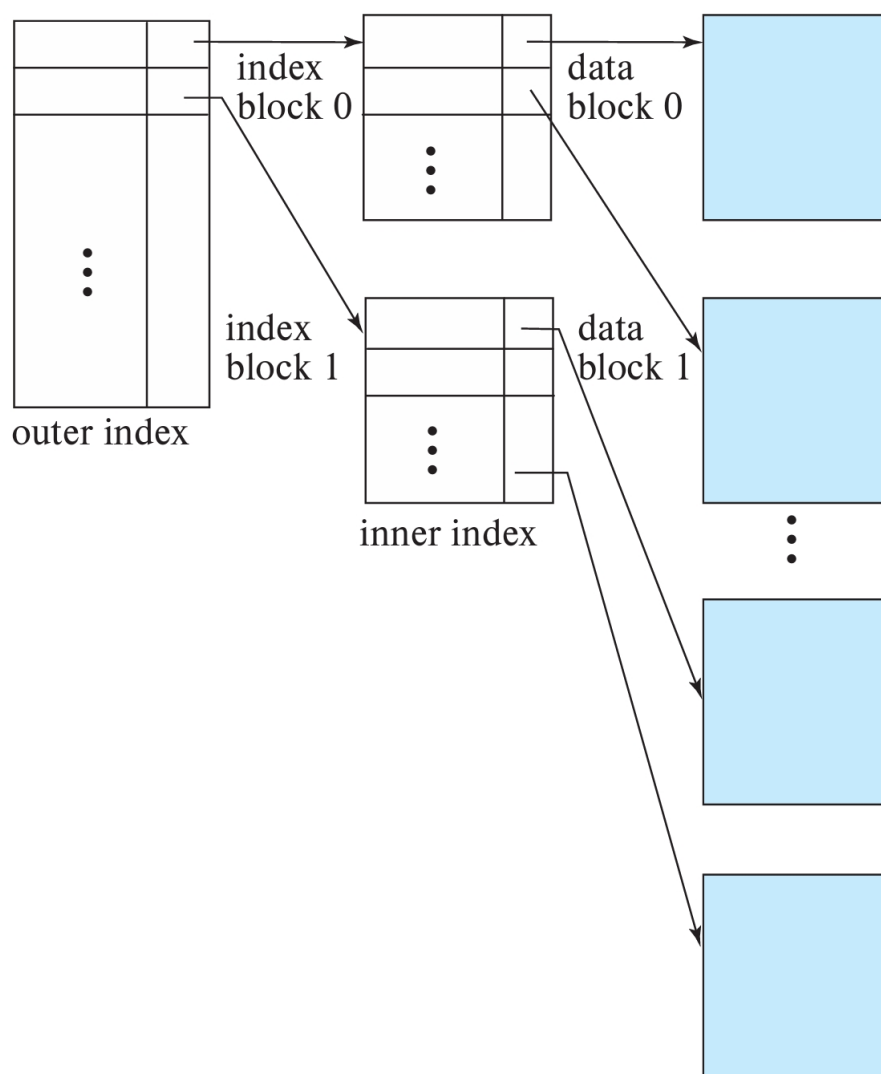


Figure 14.5 Two-level sparse index.

内层索引项总是有序排列的，这使得外层索引可以是稀疏的。

为定位一条记录，我们首先在外层索引上使用二分法找到搜索键值不超过我们所找记录的搜索键值的最大的外层索引项。

对应的二级指针指向一个内层索引块，我们扫描这个块就可找到搜索键值不超过我们所找记录的搜索键值的最大的内层索引项。

对应的一级指针指向包含我们所查找记录的文件块，我们扫描这个块就可找到所找的记录。

如果我们的文件极其庞大，甚至外层索引也可能大到不能放进主存，那么我们可以再创建另一级索引。具有两级或两级以上的索引称为**多级索引** (multi-level indices)。

4.3.4 索引更新

简单起见，我们只详细解释单级索引的更新。

多级索引的插入和删除算法是对单级索引更新算法的一个简单扩展。

在插入或删除时，系统对一级索引进行更新。

对于二级索引而言，一级索引不过是一个包含记录的文件。

因此如果一级索引发生了改变，系统就对二级索引进行更新，以此类推。

(1) 插入

系统首先用出现在待插入记录中的搜索键值执行查找，

然后根据索引是稠密的还是稀疏的来进行下一步操作：

- **稠密索引：**

如果该搜索键值并未出现在索引中，系统就在索引中适当的位置插入带有该搜索键值的索引项。

否则执行如下操作：

- 如果索引项存储的是指向具有相同搜索键值的所有记录的指针，那么系统就在索引项中增加一个指向新记录的指针。
- 否则，索引项存储一个仅指向具有相同搜索键值的第一条记录的指针，系统把待插入的记录放到具有相同搜索键值的其他记录之后。

- **稀疏索引：**

假设索引为每个块维护一个索引项。

如果系统创建了一个新的块，它会将出现在新块中的第一个搜索键值 (按照搜索键的次序) 插入索引中。

如果新插入的记录具有它所在块中的最小搜索键值，那么系统就更新指向该块的索引项；

否则，系统对索引不做任何改动。

(2) 删除

系统首先用出现在待删除记录中的搜索键值执行查找，

然后根据索引是稠密的还是稀疏的来进行下一步操作：

- **稠密索引：**

如果待删除的记录是具有这个特定搜索键值的唯一的一条记录，则系统就从索引中删除相应的索引项。

否则执行如下操作：

- 如果索引项存储的是指向具有相同搜索键值的所有记录的指针，那么系统就从索引项中删除指向待删除记录的指针。
- 否则，索引项存储一个仅指向具有该搜索键值的第一条记录的指针。如果待删除的记录是具有该搜索键值的第一条记录，系统就更新索引项，使其指向下一条记录。

- **稀疏索引：**

如果索引中并不包含具有待删除记录搜索键值的索引项，则索引不必做任何改动。

否则执行如下操作：

- 如果待删除记录是具有该搜索键值的唯一记录，则系统用下一个搜索键值 (按搜索键次序) 的索引记录来替换相应的索引记录。
- 如果下一个搜索键值已经有了一个索引项，则删除而不是替换该索引项。否则，如果该搜索键值的索引项指向待删除的记录，系统就更新索引项，使其指向具有相同搜索键值的下一条记录。

4.3.5 辅助索引

聚集索引可以是稀疏的，只存储部分搜索键值，通过顺序访问文件的一部分，就能找到带有中间搜索键值的记录。

辅助索引必须是稠密的，对每个搜索键值都有一个索引项，并且对文件中的每条记录都有一个指针。

如果辅助索引的搜索键是候选键，即**唯一性搜索键** (unique search key)，则对应的辅助索引看起来就像是稠密聚集索引，只不过索引指向的记录不是按搜索键值的顺序存放的。

如果辅助索引的搜索键不是候选键，即**非唯一性搜索键** (non-unique search key) 则可能存在多条记录具有相同搜索键值，且它们可以分布在文件的任何位置。

因此辅助索引必须包含指向所有记录的指针。

一种实现方法是辅助索引的每个 (间接) 指针都指向一个桶，该桶继而又包含指向对应记录的 (直接) 指针。例如在 instructor 文件的 dept_name 搜索键上构建的辅助索引：

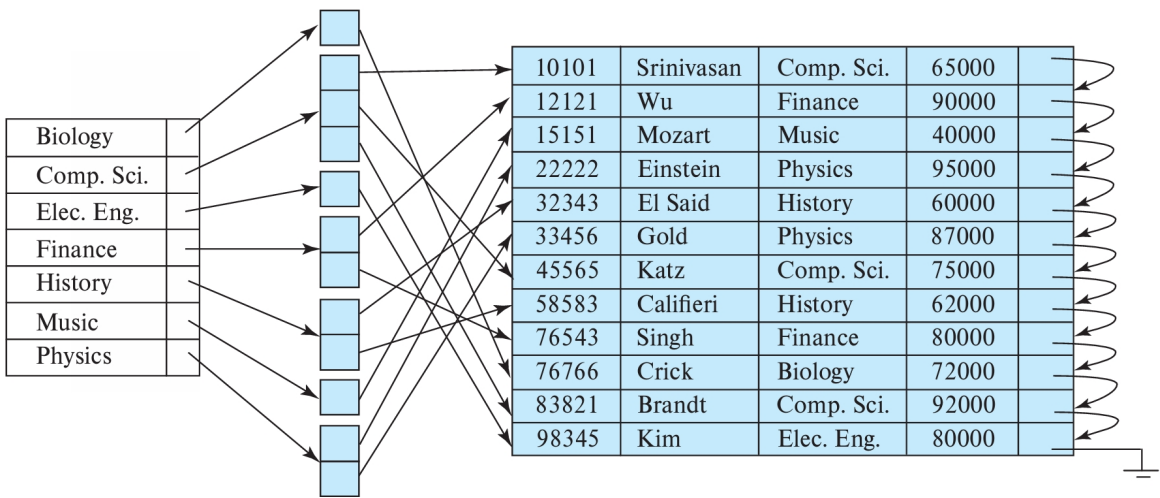


Figure 14.6 Secondary index on *instructor* file, on noncandidate key *dept_name*.

然而上述实现方式有几个缺点。

首先，由于附加的间接指针层可能需要随机 I/O 操作，因此索引访问需要花费更长的时间。

其次，如果搜索键的某些键值很少重复，那么将整个块分配给对应的桶会浪费大量的空间。

后面的课程里，我们将学习辅助索引的更有效的实现。

辅助索引提高了使用除了聚集索引的搜索键之外的键的查询性能。

但它们会给数据库的更新带来很大的开销。

我们可根据查询与更新的频率来决定哪些辅助索引是可取的。

4.4 B⁺ 树索引

索引顺序文件组织主要的缺点在于，随着文件的增大，索引查找的性能和数据顺序扫描的性能都会下降。虽然这种性能下降可以通过对文件的重新组织来弥补，但是我们不希望频繁地进行重组。

B⁺ 树索引 (B⁺-tree index) 结构采用**平衡树** (balanced tree) 结构，

其从根节点到叶节点的每条路径的长度都是相同的。

根节点有 $2 \sim n$ 个子节点，其他非叶节点有 $\lceil n/2 \rceil \sim n$ 个子节点，叶节点有

$\lceil (n-1)/2 \rceil \sim (n-1)$ 个值。

其中参数 n 对于特定的树是固定的。

一般来说，使得节点的最大大小为磁盘块大小 (通常为 4KB) 的 n 是合适的。

简单起见，我们默认 $n = 4$ ，即非叶节点可以有 $2 \sim 4$ 个子节点，每个叶节点可以有 $2 \sim 3$ 个值。

我们将看到 B^+ 树结构会增加文件插入和删除的维护开销，同时会增加空间开销。但即使对于频繁更新的文件来说，这种开销也是可接受的，因为避免了文件重组的代价。此外，由于节点有可能是半空的 (如果它们具有最少的子节点数)，因此可能造成一些空间的浪费。但考虑到 B^+ 树结构所带来的查询性能优势，这种空间浪费也是可以接受的。

4.4.1 基本结构

B^+ 树索引是一种多级索引，但是其结构不同于多级索引顺序文件。典型的 B^+ 树节点如图所示：



Figure 14.7 Typical node of a B^+ -tree.

简单起见，假设没有重复的搜索码值，即每个搜索码都是唯一的，并且最多出现在一条记录中。其中 $K_1, \dots, K_{n-1}$ 为有序存放的搜索键值 (即 $K_i < K_j$ for any $i < j$)， $P_1, \dots, P_n$ 为指针。对于叶节点，指针 $P_1, \dots, P_n$ 指向具有搜索键值 K_i 的一条文件记录。

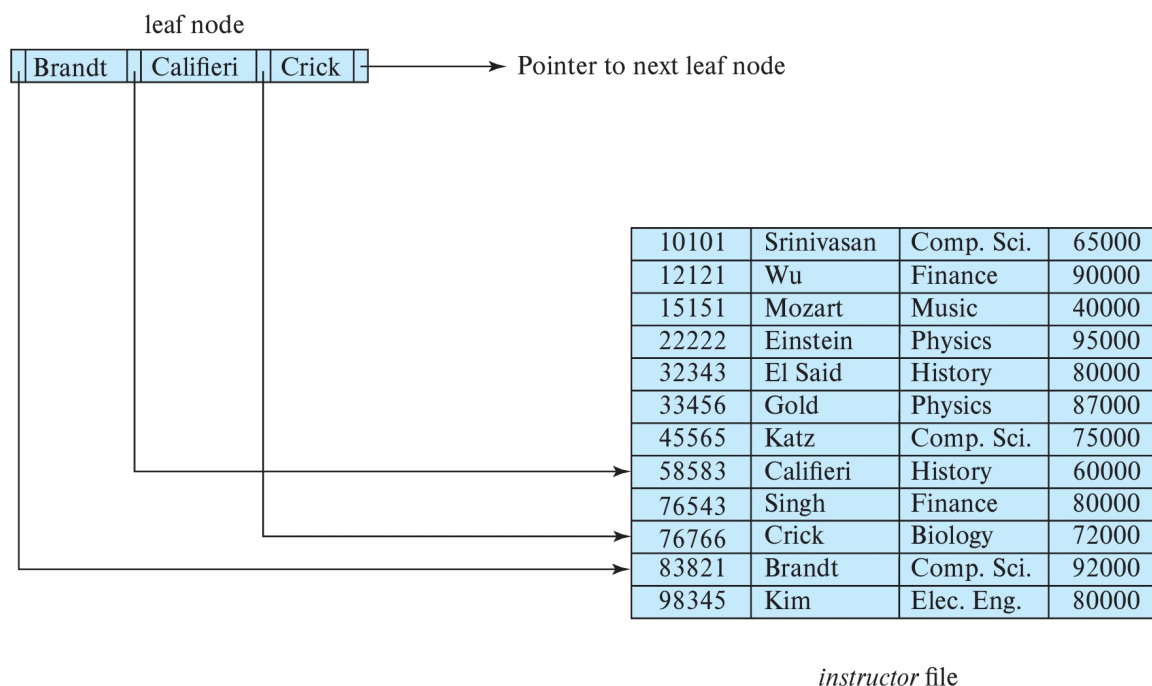


Figure 14.8 A leaf node for *instructor* B^+ -tree index ($n = 4$).

考虑稠密 B^+ 树索引，即每个搜索键值都必须出现在某个叶节点中。按照叶节点所包含的搜索键值，在叶节点上有一个线性的次序，我们用指针 P_n 将叶节点按搜索键次序链接在一起，方便对文件进行顺序处理。而 B^+ 树的**非叶节点** (nonleaf node) 形成叶节点上的一个多级 (稀疏) 索引，每个非根节点的非叶节点包含 $\lceil n/2 \rceil \sim n$ 个指针，一个节点的指针数称为该节点的**扇出** (fanout) 记一个非叶节点有 m 个指针 $P_1, P_2, \dots, P_{m-1}, P_m$

- 指针 P_1 指向子树中搜索键值 $< K_1$ 的那部分
- 指针 P_i ($i = 2, \dots, m-1$) 指向子树中搜索键值 $\in [K_{i-1}, K_i)$ 的那部分
- 指针 P_m 指向子树中搜索键值 $\geq K_m$ 的那部分

根节点与其他非叶节点不同，它包含的指针数可以少于 $\lceil n/2 \rceil$ 但除非整棵树只由一个节点组成，否则根节点必须至少包含两个指针。下图展示了 *instructor* 文件的一棵 $n = 4$ 的 B^+ 树 (省略了空指针)：

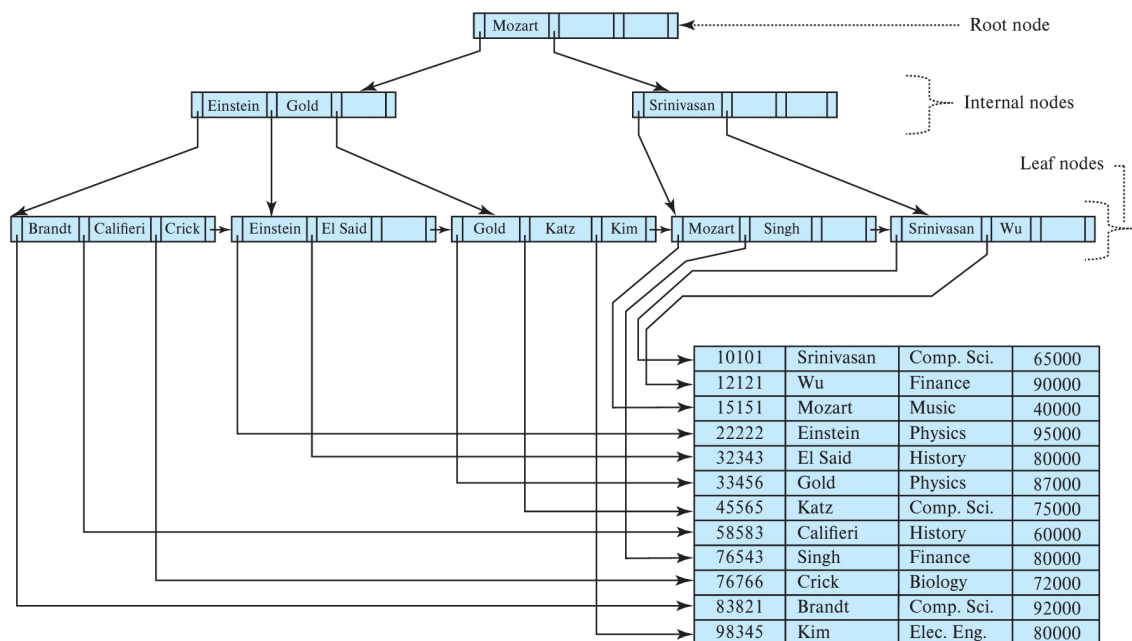


Figure 14.9 B⁺-tree for *instructor* file ($n = 4$).

下图展示了 *instructor* 文件的一棵 $n = 6$ 的 B⁺ 树 (省略了空指针):
(可以观察到这棵树的高度小于前面 $n = 4$ 的树)

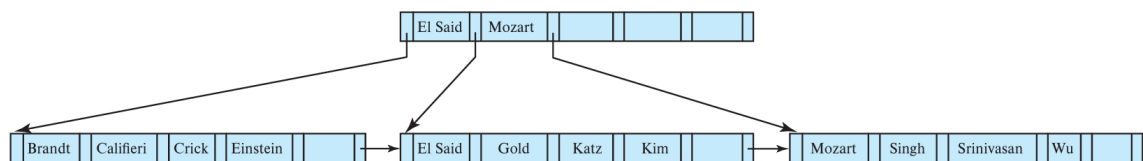


Figure 14.10 B⁺-tree for *instructor* file with $n = 6$.

上述 B⁺ 树都是平衡的, 记从根节点到叶节点的每条路径的长度都相同。
正是 B⁺ 树的平衡性保证了其对于查找、插入和删除的良好性能。

当搜索键有重复项时, 处理非唯一性搜索键的一种方式修改树结构, 使得每个搜索键在叶节点处的存储次数与它在记录中出现的次数相同, 并且每个副本指向一条记录。同时 $i < j \Rightarrow K_i < K_j$ 的条件需要修改为 $i < j \Rightarrow K_i \leq K_j$ 。但这种方式会导致内部节点的搜索键值重复, 使得插入和删除过程更加复杂且代价昂贵。另一种可选方案是存储一个带每个搜索键值的记录指针的集合 (或桶)。这种方式更为复杂, 并可能导致访问效率低下, 特别是在对于特定键值的记录指针数量非常多的情况下。

对于非唯一性搜索键, 我们还可复合其他属性来保证唯一性。例如我们希望在 *instructor* 关系的 *name* 属性上创建索引, 则我们可利用主键 *ID* 构建复合搜索键 (*name*, *ID*), 并在其上创建索引。事实上, 大多数数据库都采用这种实现。

4.4.2 查询

简单起见, 假设搜索键值具有唯一性。
考虑查询搜索键值具有给定值 v 的记录:

```

function find(v)
/* Assumes no duplicate keys, and returns pointer to the record with
 * search key value v if such a record exists, and null otherwise */
  Set C = root node
  while (C is not a leaf node) begin
    Let i = smallest number such that  $v \leq C.K_i$ 
    if there is no such number i then begin
      Let  $P_m$  = last non-null pointer in the node
      Set  $C = C.P_m$ 
    end
    else if ( $v = C.K_i$ ) then Set  $C = C.P_{i+1}$ 
    else Set  $C = C.P_i$  /*  $v < C.K_i$  */
  end
/* C is a leaf node */
if for some i,  $K_i = v$ 
  then return  $P_i$ 
  else return null ; /* No record with key value v exists*/

```

Figure 14.11 Querying a B⁺-tree.

查询从树的根节点开始，并向下遍历直到找到包含指定值的叶节点为止：

- 检查当前节点，查找最小的 *i* 使得搜索码值 $K_i \geq v$ 。
假设找到了这样的 *i*：
 - 如果 $K_i = v$ ，则将当前节点置为由 P_{i+1} 指向的节点；
 - 如果 $K_i > v$ ，则将当前节点置为由 P_i 指向的节点；

假设没找到这样的 *i*，则意味着 $v > K_{m-1}$ ，其中 P_m 是节点中的最后一个非空指针。

在这种情况下，将当前节点置为由 P_m 指向的节点。

重复上述过程，向下遍历树直至到达叶节点为止。

如果搜索键值 *v* 对应某个叶节点，则 *find*(*v*) 得到指向对应记录的指针。

否则 *find*(*v*) 返回空值，代表查询失败。

考虑查询搜索键值在区间 $[l, u]$ 中的记录。

findRange(*l*, *u*) 首先以类似于 *find*(*l*) 的方式遍历至一个叶节点 (其搜索键值 $\geq l$)

然后按顺序遍历收集在区间 $[l, u]$ 中的所有记录。

```

function findRange(lb, ub)
/* Returns all records with search key value V such that  $lb \leq V \leq ub$ . */
  Set resultSet = {};
  Set C = root node
  while (C is not a leaf node) begin
    Let i = smallest number such that  $lb \leq C.K_i$ 
    if there is no such number i then begin
      Let  $P_m$  = last non-null pointer in the node
      Set  $C = C.P_m$ 
    end
    else if ( $lb = C.K_i$ ) then Set  $C = C.P_{i+1}$ 
    else Set  $C = C.P_i$  /*  $lb < C.K_i$  */
  end
/* C is a leaf node */
  Let i be the least value such that  $K_i \geq lb$ 
  if there is no such i
    then Set i = 1 + number of keys in C; /* To force move to next leaf */
  Set done = false;
  while (not done) begin
    Let n = number of keys in C.
    if ( $i \leq n$  and  $C.K_i \leq ub$ ) then begin
      Add  $C.P_i$  to resultSet
      Set  $i = i + 1$ 
    end
    else if ( $i \leq n$  and  $C.K_i > ub$ )
      then Set done = true;
    else if ( $i > n$  and  $C.P_{n+1}$  is not null)
      then Set  $C = C.P_{n+1}$ , and  $i = 1$  /* Move to next leaf */
    else Set done = true; /* No more leaves to the right */
  end
return resultSet;

```

Figure 14.12 Range query on a B⁺-tree.

4.4.3 更新

查找是插入和删除的基础，我们可以使用类似的逻辑找到待插入或待删除的指针。但插入和删除要比查找更加复杂，因为一个节点可能由于插入而变得过大（多于 n 个指针）导致需要拆分，或由于删除而变得过小（少于 $\lceil n/2 \rceil$ 个指针）而需要合并。此外，当节点被拆分或合并时，我们必须保持 B⁺ 树的平衡性。

(1) 插入

考虑在以下 B⁺ 树中插入一条 name 值为 "Adam" 的记录：

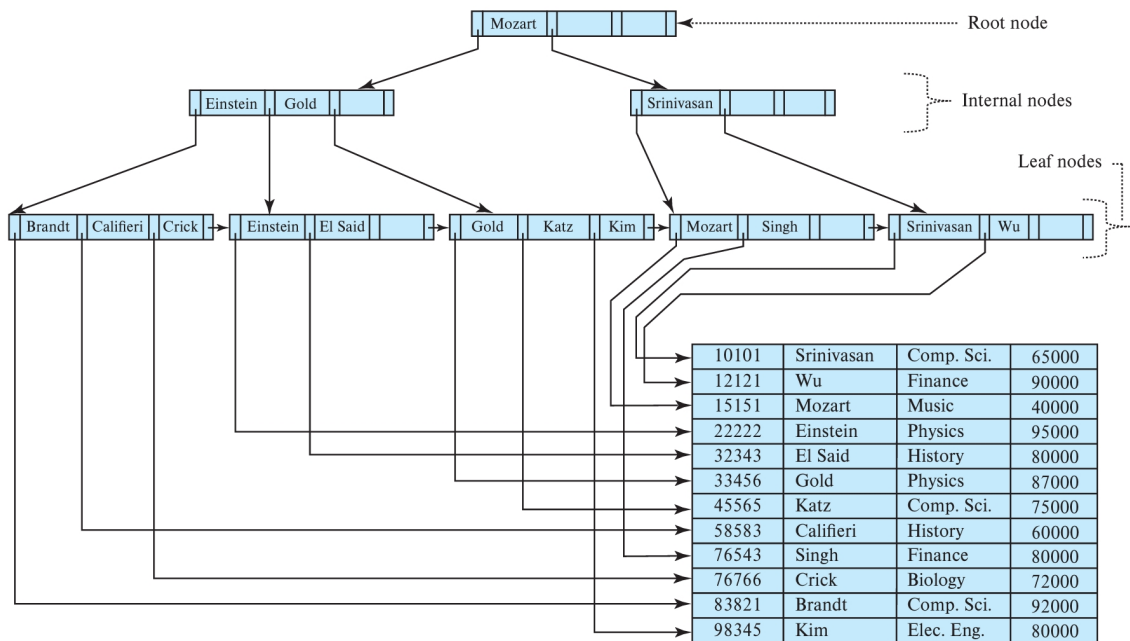


Figure 14.9 B⁺-tree for *instructor* file ($n = 4$).

我们发现待插入记录应该出现在包含 "Brandt"、"Califieri" 和 "Crick" 的叶节点中。但该叶节点中已没有插入搜索键值 "Adams" 所需的空间。因此我们需要将其拆分成两个节点：

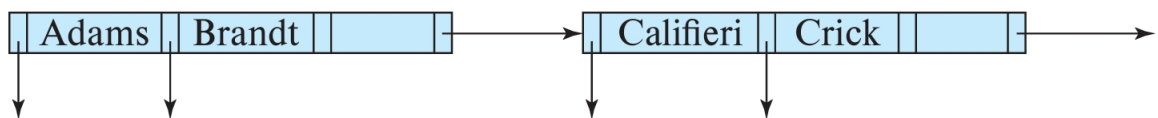


Figure 14.13 Split of leaf node on insertion of "Adams".

更新后的 B⁺ 树如下：

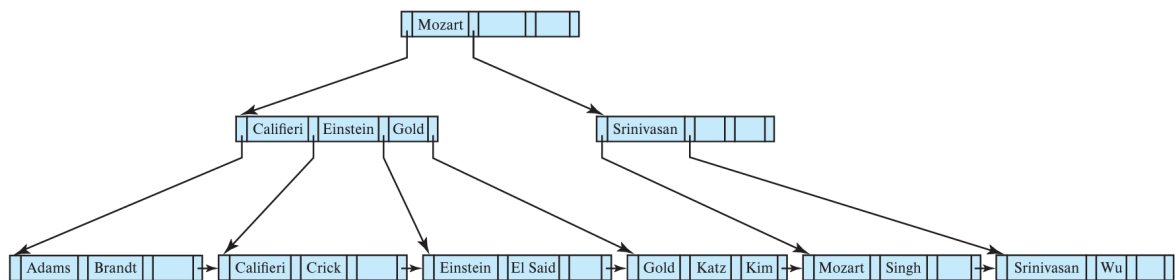


Figure 14.14 Insertion of "Adams" into the B⁺-tree of Figure 14.9.

在上述示例中，被拆分节点的父节点中有空间来容纳新项，所以无需进一步拆分父节点。

考虑继续插入一条 name 值为 "Lamport" 的记录。我们发现待插入记录应该出现在包含 "Gold"、"Katz" 和 "Kim" 的叶节点中。但该叶节点中已没有插入搜索键值 "Lamport" 所需的空间，因此需要拆分。同时父节点也需要拆分，子指针将在原节点和新创建的节点之间进行划分：

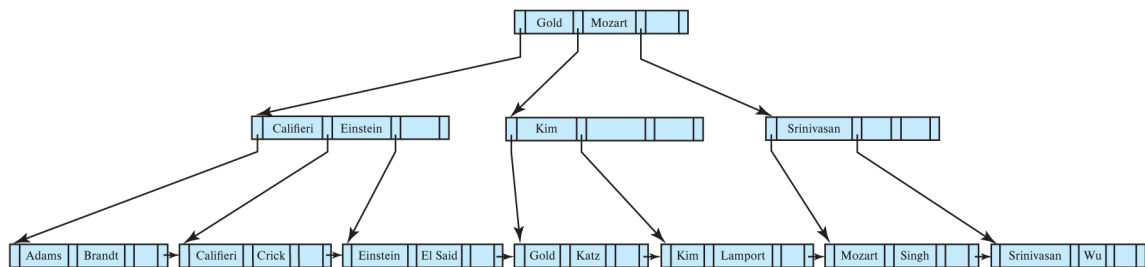


Figure 14.15 Insertion of "Lamport" into the B⁺-tree of Figure 14.14.

在最坏情况下，从待拆分的叶节点到根节点的路径上的所有节点都必须被拆分。

如果根节点本身也被拆分，那么 B⁺ 树就会变得更深。

插入算法 insert() 如下：

(插入过程中，节点临时扩张，然后使用辅助函数来拆分 "过满" 的节点)

```

procedure insert(value  $K$ , pointer  $P$ )
  if (tree is empty) create an empty leaf node  $L$ , which is also the root
  else Find the leaf node  $L$  that should contain key value  $K$ 
  if ( $L$  has less than  $n - 1$  key values)
    then insert_in_leaf ( $L, K, P$ )
  else begin /*  $L$  has  $n - 1$  key values already, split it */
    Create node  $L'$ 
    Copy  $L.P_1 \dots L.K_{n-1}$  to a block of memory  $T$  that can
      hold  $n$  (pointer, key-value) pairs
    insert_in_leaf ( $T, K, P$ )
    Set  $L'.P_n = L.P_n$ ; Set  $L.P_n = L'$ 
    Erase  $L.P_1$  through  $L.K_{n-1}$  from  $L$ 
    Copy  $T.P_1$  through  $T.K_{\lceil n/2 \rceil}$  from  $T$  into  $L$  starting at  $L.P_1$ 
    Copy  $T.P_{\lceil n/2 \rceil + 1}$  through  $T.K_n$  from  $T$  into  $L'$  starting at  $L'.P_1$ 
    Let  $K'$  be the smallest key-value in  $L'$ 
    insert_in_parent( $L, K', L'$ )
  end
  
```

Figure 14.16 Insertion of entry in a B⁺-tree.

辅助函数 insert_in_leaf() 和 insert_in_parent() 如下：

```

procedure insert_in_leaf (node L, value K, pointer P)
    if ( $K < L.K_1$ )
        then insert  $P, K$  into  $L$  just before  $L.P_1$ 
    else begin
        Let  $K_i$  be the highest value in  $L$  that is less than or equal to  $K$ 
        Insert  $P, K$  into  $L$  just after  $L.K_i$ 
    end
procedure insert_in_parent(node N, value K', node N')
    if ( $N$  is the root of the tree)
        then begin
            Create a new node  $R$  containing  $N, K', N'$   /*  $N$  and  $N'$  are pointers */
            Make  $R$  the root of the tree
        return
    end
    Let  $P = \text{parent}(N)$ 
    if ( $P$  has less than  $n$  pointers)
        then insert  $(K', N')$  in  $P$  just after  $N$ 
    else begin /* Split  $P$  */
        Copy  $P$  to a block of memory  $T$  that can hold  $P$  and  $(K', N')$ 
        Insert  $(K', N')$  into  $T$  just after  $N$ 
        Erase all entries from  $P$ ; Create node  $P'$ 
        Copy  $T.P_1 \dots T.P_{\lfloor (n+1)/2 \rfloor}$  into  $P$ 
        Let  $K'' = T.K_{\lfloor (n+1)/2 \rfloor}$ 
        Copy  $T.P_{\lfloor (n+1)/2 \rfloor + 1} \dots T.P_{n+1}$  into  $P'$ 
        insert_in_parent( $P, K'', P'$ )
    end

```

Figure 14.17 Subsidiary procedures for insertion of entry in a B^+ -tree.

(2) 删除

考虑在以下 B^+ 树中删除一条 name 值为 "Srinivasan" 的记录:

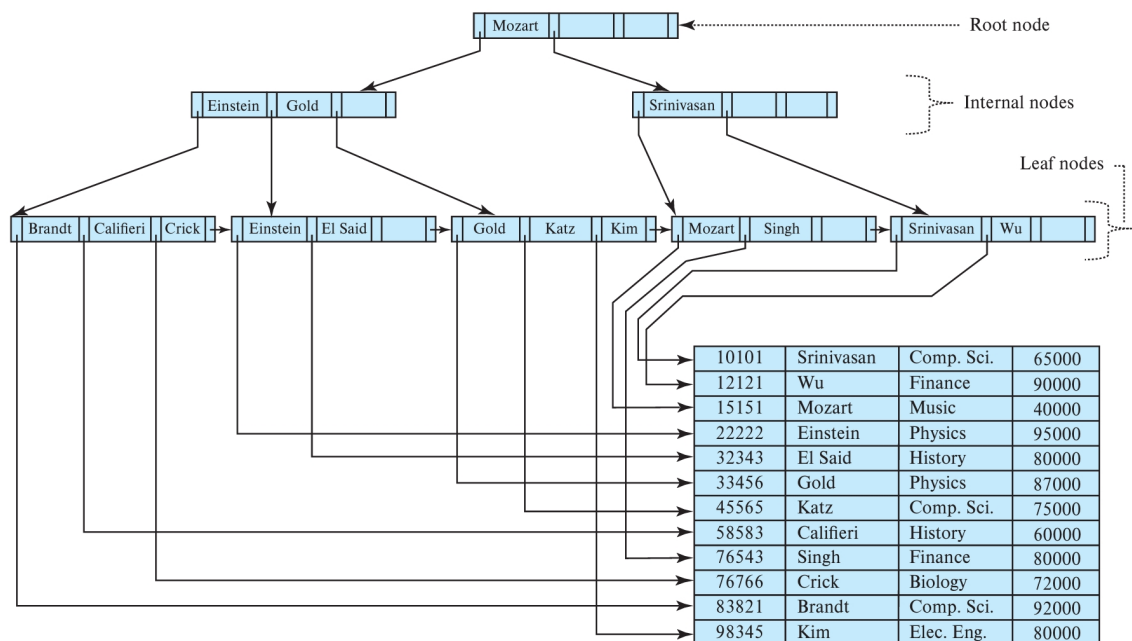


Figure 14.9 B⁺-tree for *instructor* file ($n = 4$).

我们发现待删除记录出现在包含 "Srinivasan" 和 "Wu" 的叶节点中。

但该叶节点把 "Srinivasan" 的项删除时，叶节点就只剩下一个 "Wu" 的项了 ($1 < \lceil n/2 \rceil = 2$)

于是我们必须将该节点与一个兄弟节点合并，或者在节点之间重分配项，以保证每个节点至少是半满的。

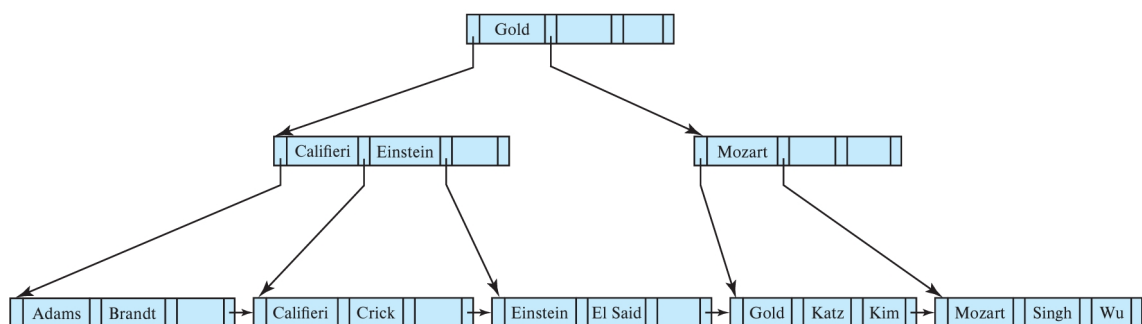


Figure 14.18 Deletion of "Srinivasan" from the B⁺-tree of Figure 14.14.

接下来我们继续删除 "Singh" 和 "Wu"，并执行重新分配：

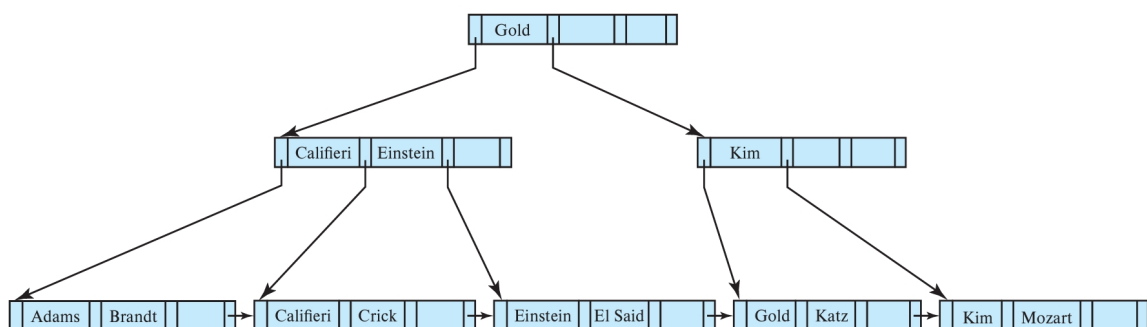


Figure 14.19 Deletion of "Singh" and "Wu" from the B⁺-tree of Figure 14.18.

最后我们删除 "Gold"，合并后根节点仅剩下一个子节点，且没有搜索键值，

违反了根节点必须至少有两个孩子的要求，

于是删除根节点，其唯一的子节点成为根节点，树的深度减 1：

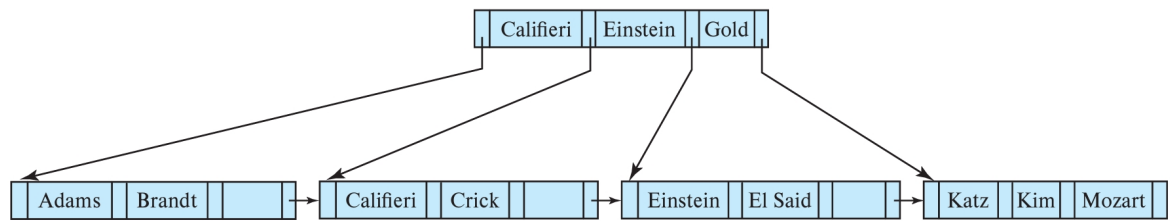


Figure 14.20 Deletion of “Gold” from the B⁺-tree of Figure 14.19.

一般地，为了在 B⁺ 树中删除一个值，我们在该值上执行查找并进行删除。

如果节点太小，我们把它从它的父节点中删除。

这种删除会导致删除算法的递归应用，直至到达树的根节点。

在删除后父节点要保持足够满，否则要进行重新分配。

从 B⁺ 树进行删除操作的 delete() 如下：

```

procedure delete(value K, pointer P)
    find the leaf node L that contains (K, P)
    delete_entry(L, K, P)

procedure delete_entry(node N, value K, pointer P)
    delete (K, P) from N
    if (N is the root and N has only one remaining child)
    then make the child of N the new root of the tree and delete N
    else if (N has too few values/pointers) then begin
        Let N' be the previous or next child of parent(N)
        Let K' be the value between pointers N and N' in parent(N)
        if (entries in N and N' can fit in a single node)
            then begin /* Coalesce nodes */
                if (N is a predecessor of N') then swap_variables(N, N')
                if (N is not a leaf)
                    then append K' and all pointers and values in N to N'
                    else append all (Ki, Pi) pairs in N to N'; set N'.Pn = N.Pn
                delete_entry(parent(N), K', N); delete node N
            end
        else begin /* Redistribution: borrow an entry from N' */
            if (N' is a predecessor of N) then begin
                if (N is a nonleaf node) then begin
                    let m be such that N'.Pm is the last pointer in N'
                    remove (N'.Km-1, N'.Pm) from N'
                    insert (N'.Pm, K') as the first pointer and value in N,
                        by shifting other pointers and values right
                    replace K' in parent(N) by N'.Km-1
                end
            else begin
                let m be such that (N'.Pm, N'.Km) is the last pointer/value
                    pair in N'
                remove (N'.Pm, N'.Km) from N'
                insert (N'.Pm, N'.Km) as the first pointer and value in N,
                    by shifting other pointers and values right
                replace K' in parent(N) by N'.Km
            end
        end
        else ... symmetric to the then case ...
    end
end

```

Figure 14.21 Deletion of entry from a B⁺-tree.

4.4.4 变体: B 树索引

B 树索引和 B⁺ 树索引是类似的，主要区别在于 B 树去除了搜索键值的冗余存储。

在 B⁺ 树中，每个搜索键值都出现在某些叶节点中，有的还在非叶节点中重复出现。

例如在以下 B⁺ 树中，搜索键 "Einstein" 和 "Gold" 在非叶节点和叶节点中都出现了。

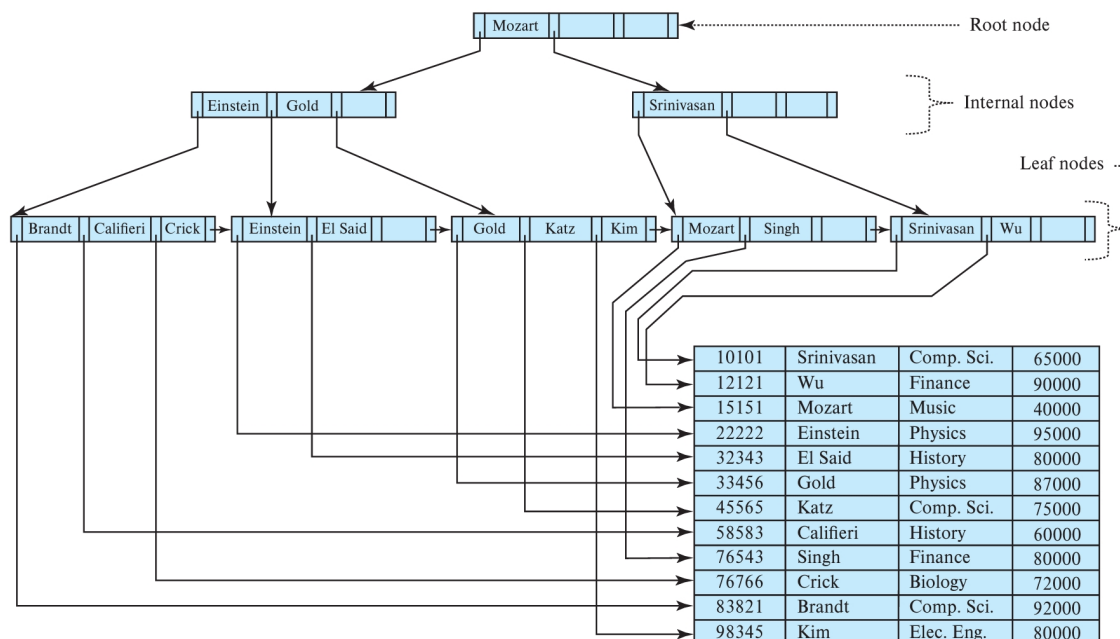


Figure 14.9 B⁺-tree for *instructor* file ($n = 4$).

与 B⁺ 树不同，B 树只允许搜索键值出现一次 (如果它们是唯一的):

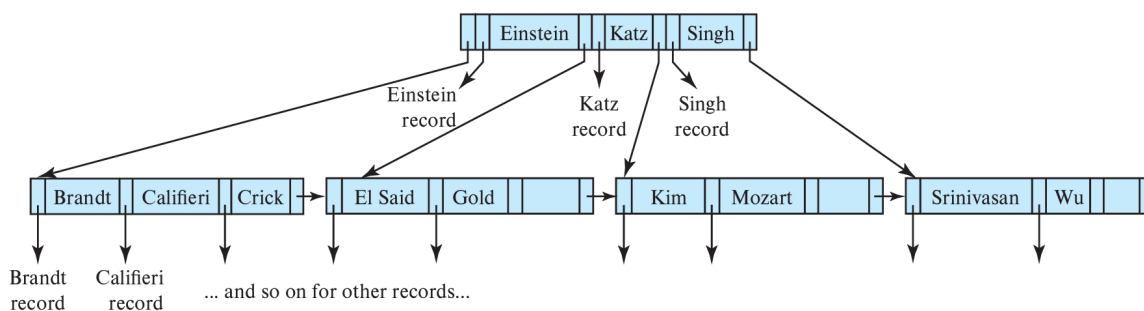


Figure 14.23 B-tree equivalent of B⁺-tree in Figure 14.9.

一种广义的 B 树节点如下图所示:



(a)



(b)

Figure 14.24 Typical nodes of a B-tree. (a) Leaf node. (b) Nonleaf node.

其中 B 树的非叶节点包含 $m - 1$ 个指向记录的指针 B_i 和 m 个指向子节点的指针 P_i . 在 B 树中进行一次查找所访问的节点数取决于搜索码所在的位置. B^+ 树上的一次查找需要遍历从根节点到某个叶节点的一条路径, 而在 B 树中有时在到达叶节点之前就能找到想要的值. 然而 B 树中存储在叶子层的键大约是存储在非叶子层的键的 n 倍, 并且由于 n 一般很大, 因此较快找到特定值的优势会相对小一些. 而且比起 B^+ 树来, B 树的非叶节点中出现的搜索键较少, 这意味着 B 树的扇出较少, 因而比相应的 B^+ 树的深度可能要大. 因此在 B 树中查找某些值可能会更快, 而查找另一些值则可能会更慢, 虽然总的来说查找时间仍然与搜索键数量的对数成正比.

B 树中的删除更加复杂. 在 B^+ 树中, 被删除的项总会出现于叶节点中. 在 B 树中, 被删除的项可能出现在非叶节点中, 必须从包含被删除项的节点所在的子树中选择正确的值来作为替代. 具体来说, 如果搜索键 K_i 被删除, 则出现在指针 P_{i+1} 的子树中的最小搜索键必须被移到原先被 K_i 所占用的字段中. 如果叶节点包含的项太少, 还需采取进一步的操作. 与此相比, B 树中的插入只比 B^+ 树中的插入稍微复杂一点.

B 树在空间上的优势对于大型索引而言意义不大, 而且可能增加查询和更新的代价. 因此几乎所有的数据库系统实现都采用 B^+ 树索引.

4.5 散列索引

散列 (hashing) 是在主存中构建索引的一种广泛使用的技术. 我们称**桶** (bucket) 来表示可以存储若干条记录的存储单元. 对于内存中的散列索引, 桶可以是索引项或记录的链表, 而桶的集合只是一个指针数组. 对于基于磁盘的索引, 桶可以是磁盘块的链表. 在**散列文件组织** (hash file organization) 中, 桶存储实际的记录, 而不是记录的指针. 这种结构只对驻留在磁盘上的数据才有意义.

记 $\mathcal{K}$ 为所有搜索键值的集合, 记 $\mathcal{B}$ 为所有桶地址的集合. 散列函数 $h : \mathcal{K} \mapsto \mathcal{B}$ 是一个单射. 为插入一条带有搜索键 K_i 的记录, 我们计算 $h(K_i)$ 得到对应桶的地址, 并插入该桶. 如果该桶没有足够的空间, 即发生**桶溢出** (bucket overflow), 那么我们使用**溢出桶** (overflow bucket) 来处理桶溢出. 一个给定桶的所有溢出桶都链接在一起并存放在链表中:

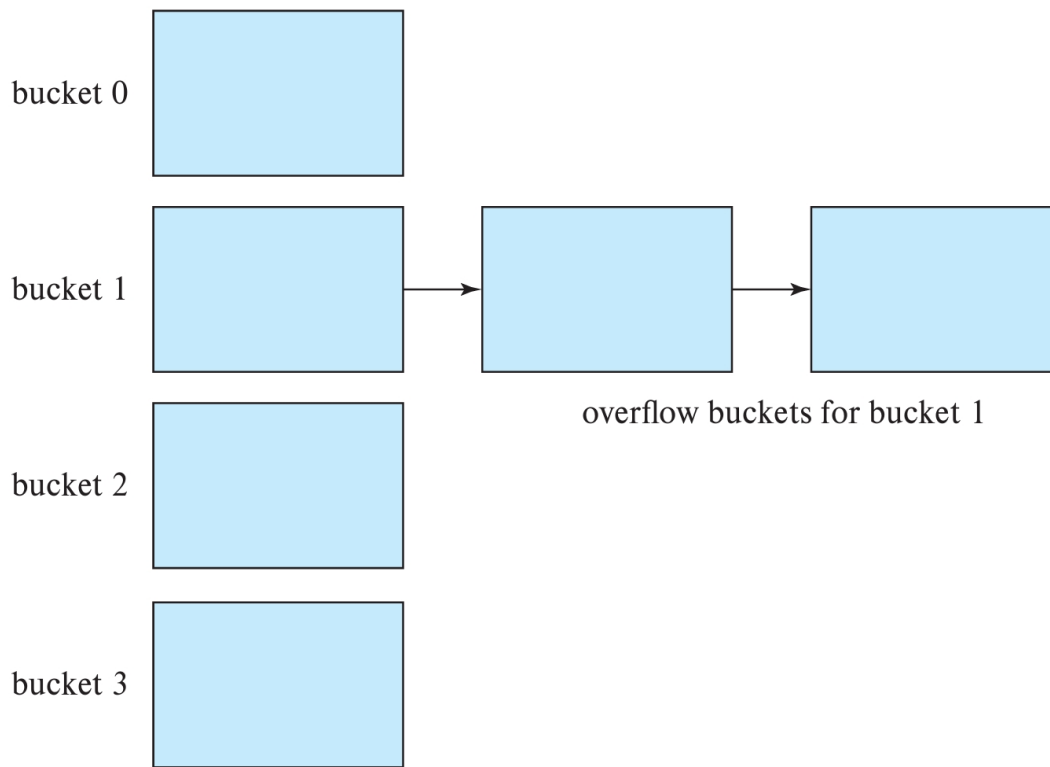


Figure 14.25 Overflow chaining in a disk-based hash structure.

为避免在其他桶仍有大量自由空间的时候仍有部分桶溢出的问题，我们应当仔细选择散列函数，来将此问题的可能性降至最低，保证键在桶之间的分布是均匀的。

对于**静态散列** (static hashing) 来说，桶的数量可设为 $(N/N_{\text{bucket}}) \times (1 + d)$

其中 N 代表记录总数， N_{bucket} 代表每个桶能容纳的记录数， d 是一个避让因子 (通常为 0.2)

静态散列的一个问题是：我们需要知道有多少条记录将存储在索引中。

如果随着时间的推移添加了大量的记录，导致记录远远多于桶，

则查找将不得不搜索存储在单个桶中或者在一个或多个溢出桶中的大量记录，并因而变得低效。

为了处理这个问题，可以使用可增长数量的桶来重建散列索引。

例如如果记录数变为桶数的两倍，则可以使用比以前多一倍的桶来重建索引。

但是重建索引的缺点是，如果记录数很多，则可能需要花费很长时间来重建，从而导致正常处理的中断。

更好的解决方法是**动态散列** (dynamic hashing)，

例如线性散列 (linear hashing) 和可扩展散列 (extendable hashing)。

The End